

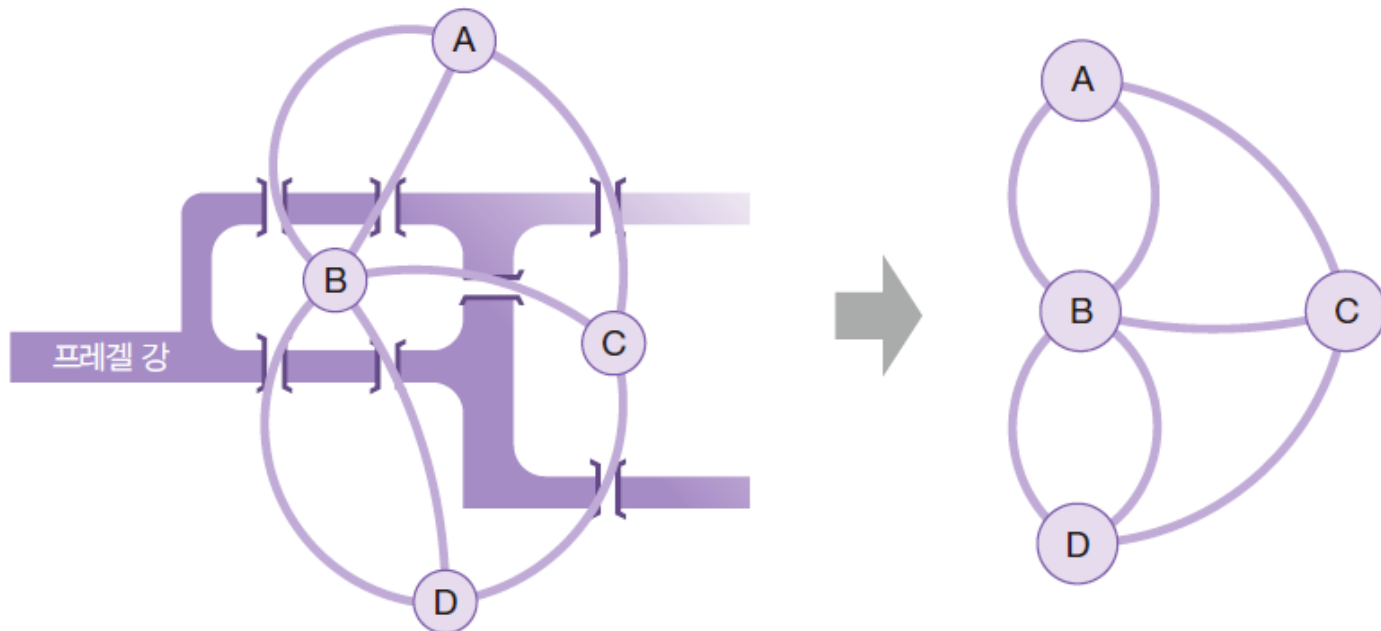
그래프 기초

그래프의 개요

- 그래프란?
 - 트리와 더불어 컴퓨터로 응용문제를 해결하기 위해 널리 사용되는 자료구조 중의 하나
 - 컴퓨터 네트워크 망의 진단
 - 내비게이션의 지도 안내
 - 프로젝트 진도 관리

그래프의 개요

- 그래프의 탄생 배경: 오일러의 문제 해결 도구
 - 레온하르트 오일러 (Leonhard Euler)
 - 육지를 정점(Vertex) 으로, 각 육지를 잇는 다리를 간선(Edge)으로 가정



그래프의 개요

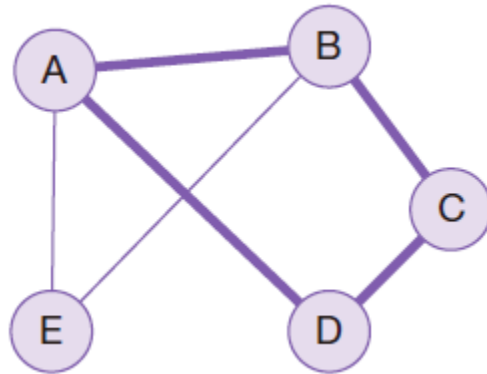
- 그래프의 정의

- 그래프는 '정점의 모음'과 이 정점을 잇는 '간선의 모음'이 결합한 것
- 정점의 집합을 V , 간선의 집합을 E , 그래프를 G 라고 했을 때 $G = (V, E)$



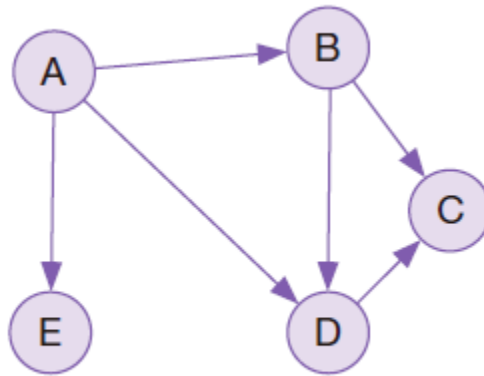
그래프의 개요

- 사이클(Cycle) - 정점 하나를 두 번 이상 거치도록 되어 있는 경로
- 사이클은 트리에서 볼 수 없는 특징

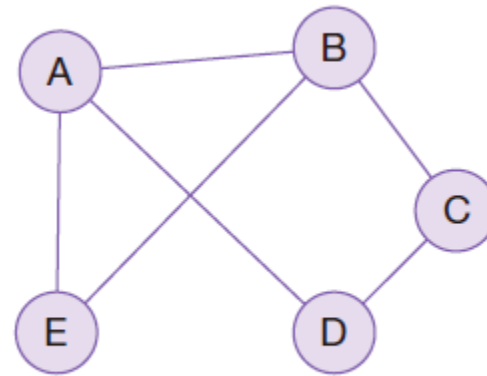


그래프의 개요

- 방향성 그래프(Directed Graph)와 무방향성 그래프(Undirected Graph)



방향성 그래프

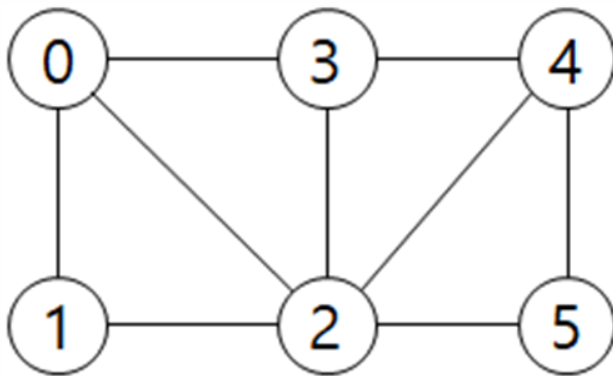


무방향성 그래프

그래프의 개요

- 그래프의 표현 방법
 - 인접행렬(adjacency matrix)
 - 인접리스트(adjacency list)

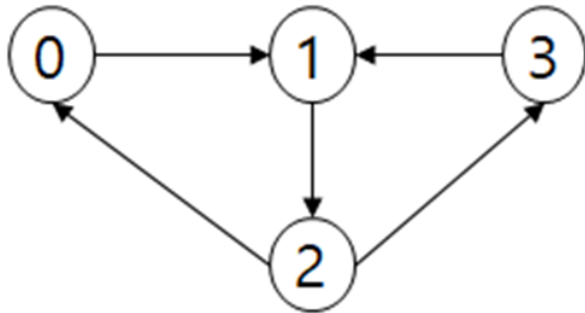
인접행렬 그래프



	0	1	2	3	4	5
0	0	1	1	1	0	0
1	1	0	1	0	0	0
2	1	1	0	1	1	1
3	1	0	1	0	1	0
4	0	0	1	1	0	1
5	0	0	1	0	1	0

(그림 9.1) 무방향 그래프의 인접행렬 표현 예

인접행렬 그래프



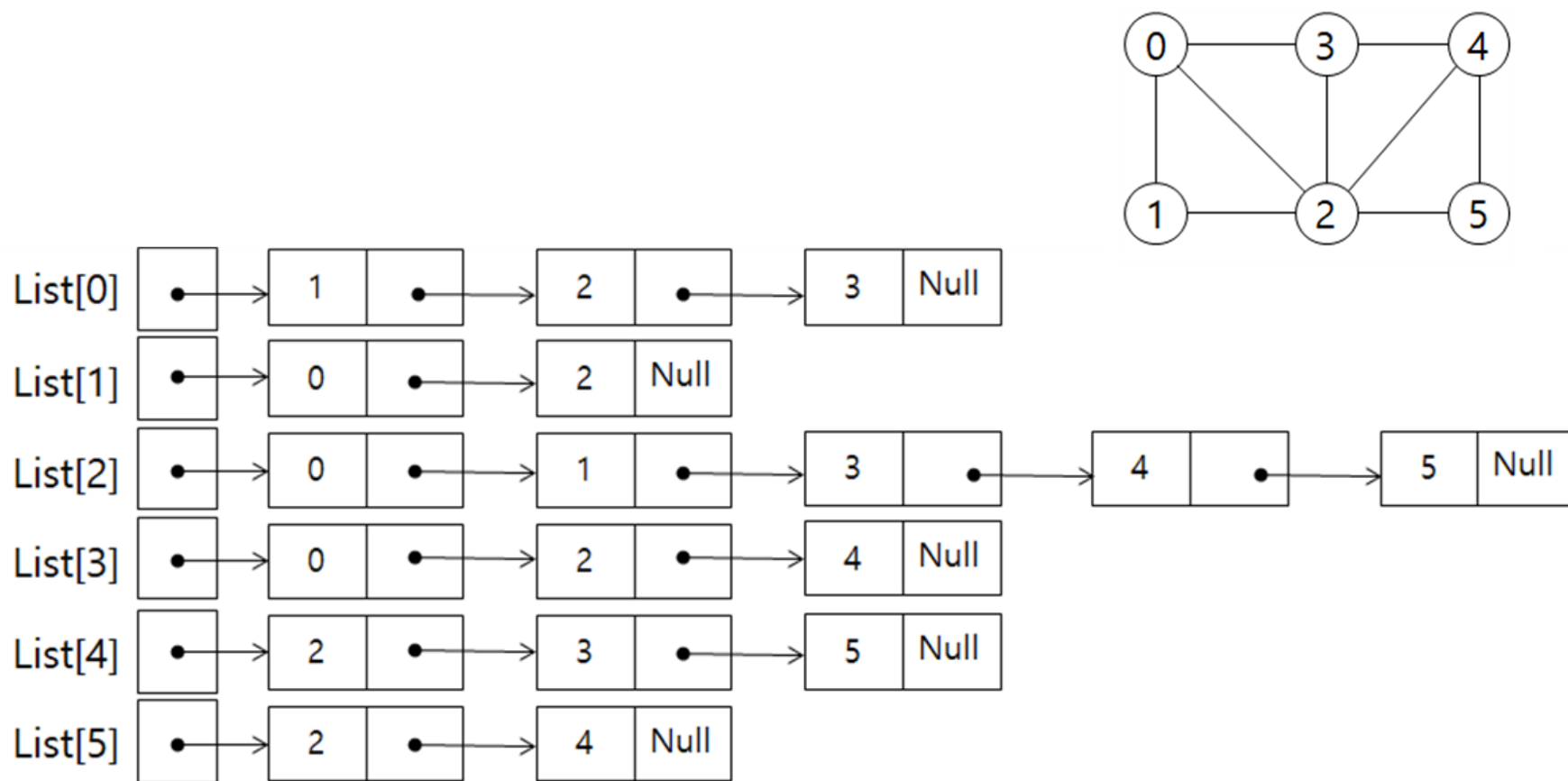
	0	1	2	3
0	0	1	0	0
1	0	0	1	0
2	1	0	0	1
3	0	1	0	0

(그림 9.2) 방향 그래프의 인접행렬 표현 예

연결리스트 그래프

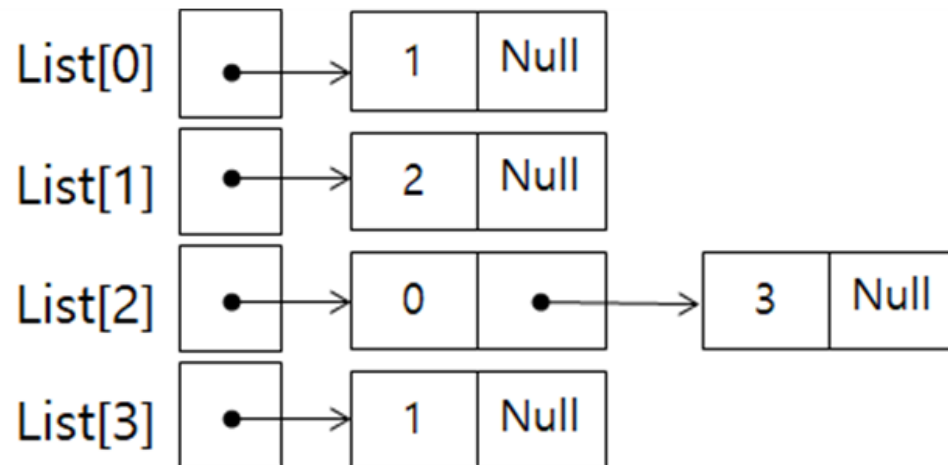
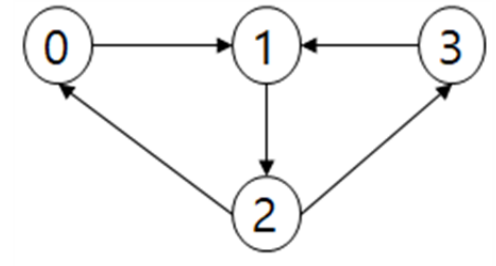
```
define No_of_Vertex 5
struct G_node
    { vertex
      link } G_node
G_node List[No_of_Vertex]
```

연결리스트 그래프



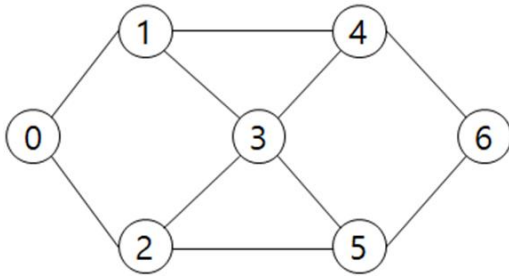
(그림 9.3) (그림 9.1)의 인접리스트 표현 예

연결리스트 그래프



(그림 9.4) (그림 9.2)의 인접리스트 표현 예

인접행렬 기반 그래프



(그림 9.5) 7개의 정점을 갖는 무방향 그래프의 예

	0	1	2	3	4	5	6
0							
1							
2							
3							
4							
5							
6							

인접행렬 기반 그래프

- 무방향 그래프

1. Algorithm ADJ_create(g_file, n)
2. Input: 무방향 그래프 간선들이 저장된 파일(g_file), 정점의 개수(n)
3. Output: 주어진 그래프의 인접행렬
4. 인접행렬 Adj_mat 배열을 생성하고 0으로 초기화
5. open g_file
6. while(!end_of_file) do {
7. read i, j
8. Adj_mat[i, j] <- 1
9. Adj_mat[j, i] <- 1 }
10. close g_file
11. return Adj_mat

그래프

- 무방향 인접행렬 차수 계산

1. Algorithm Adj_degree(Adj_mat, n)
2. Input: 무방향 그래프의 인접행렬(Adj_mat), 정점의 개수(n)
3. Output: 각 정점의 차수
4. for i <- 0 to n-1 do {
5. degree <- 0
6. for j <- 0 to n-1 do
7. degree <- degree + Adj_mat[i, j]
8. print i, degree
9. }
10. end

그래프

- 방향 인접행렬 차수 계산

〈정점 i 의 진입차수〉

```
degree ← 0
```

```
for j ← 0 to n-1 do
```

```
    degree ← degree + Adj_mat[j, i]
```

〈정점 i 의 진출차수〉

```
degree ← 0
```

```
for j ← 0 to n-1 do
```

```
    degree ← degree + Adj_mat[i, j]
```


그래프

- 인접행렬 -> 연결리스트 변환

1. Algorithm Adj_insert(List, i, j)
2. Input: 인접리스트의 헤더 포인터 배열(List), 간선 (i, j)
3. Output: List[i]에 정점 j를 삽입
4. node <- malloc(sizeof(G_node))
5. node.vertex <- j
6. node.link <- List[i]
7. List[i] <- node
8. return List

그래프

- 인접행렬 -> 연결리스트 변환

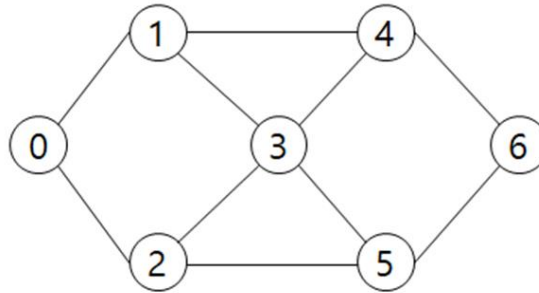
1. Algorithm Adjmat2list(Adj_mat, n)
2. Input: 그래프의 인접행렬(Adj_mat), 정점의 개수(n)
3. Output: 인접행렬을 인접리스트로 변환한 결과
4. List 헤더 포인터 배열을 Null로 초기화
5. for i <- 0 to n-1 do {
6. for j <- 0 to n-1 do
7. if (Adj_mat[i, j] = 1)
8. List <- Adj_insert(List, i, j)
9. }
10. return List

깊이우선탐색(DFS)

- **깊이 우선 탐색(Depth First Search)**

- ➊ 시작 정점을 밟은 후 이 정점을 '방문했음'으로 표시
- ➋ 이 정점과 이웃 정점(인접 정점) 중에 아직 방문하지 않은 곳을 선택하여 이를 시작 정점으로 삼고 다시 깊이 우선 탐색을 시작. 즉, 단계 ➊을 다시 수행
- ➌ 더 이상 방문하지 않은 이웃 정점이 없으면 이전 정점으로 돌아가 단계 ➋를 수행
- ➍ 이전 정점으로 돌아가도 더 이상 방문할 이웃 정점이 없다면 그래프의 모든 정점을 방문했다는 뜻이므로 탐색을 종료

깊이우선탐색(DFS)



	0	1	2	3	4	5	6
visited[]							

깊이우선탐색(DFS)

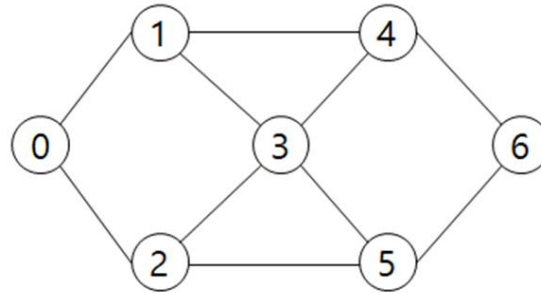
1. Algorithm Graph_dfs(List, v)
2. Input: 그래프의 인접리스트(List), 시작 정점 v
3. Output: DFS 방식으로 방문한 그래프의 정점들
4. w를 List[v]로 초기화 // 최초 실행 시만 수행
5. visited[v] <- 1
6. print v
7. while (w != Null) do {
8. temp <- w.vertex
9. if (visited[temp] = 0)
10. Graph_dfs(List, temp)
11. w <- w.link
12. }
13. end

너비우선탐색(BFS)

너비 우선 탐색(Breadth First Search)

- ❶ 시작 정점을 '방문했음'으로 표시하고 큐에 삽입
- ❷ 큐로부터 정점을 제거. 제거한 정점의 인접 정점 중에서 아직 방문하지 않은 곳을 '방문했음'으로 표시하고 큐에 삽입
- ❸ 큐가 비면 탐색이 완료. 따라서 큐가 빌 때까지 단계 ❷의 과정을 반복

너비우선탐색(BFS)



	0	1	2	3	4	5	6
visited[]							
queue[]							

너비우선탐색(BFS)

1. Algorithm Graph_bfs(List, v)
2. Input: 그래프의 인접리스트(List), 시작 정점 v
3. Output: BFS 방식으로 방문한 그래프의 정점들
4. init Queue
5. visited[v] <- 1
6. print v
7. Queue_enqueue(v)
8. loop (!Queue_empty()) do {
9. v <- Queue_dequeue()
10. w <- List[v]
11. while (w != Null) do {
12. if (visited[w.vertex] = 0) {
13. visited[w.vertex] <- 1
14. print w.vertex
15. Queue_enqueue(w.vertex) }
16. w <- w.link }
17. }
18. end

위상 정렬

위상 정렬(Topological Sort)

조건

- 그래프에 방향성이 있음
- 그래프 내에 사이클이 없음

위상 정렬

BFS를 이용한 위상 정렬

- 진입 차수를 활용한 정렬

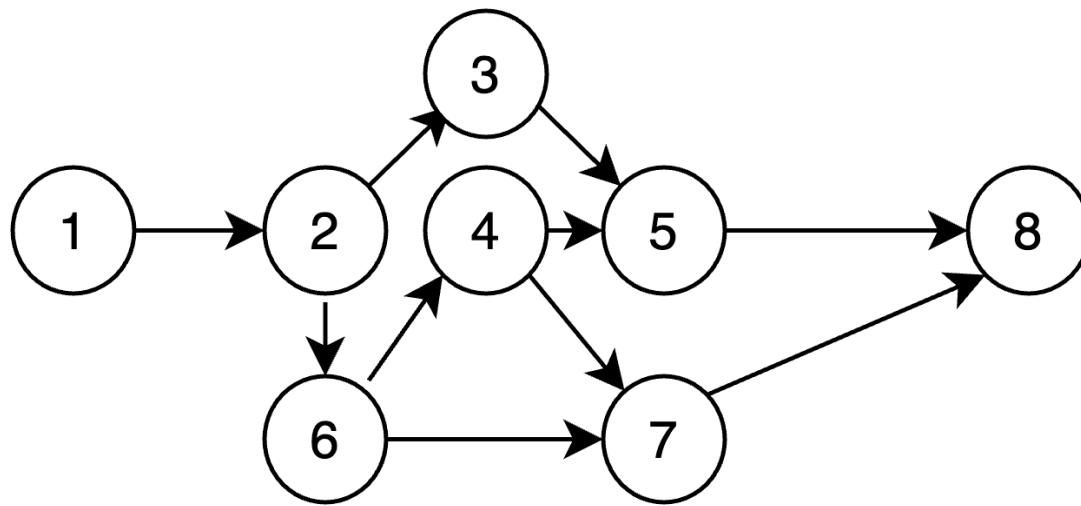
- ① 모든 정점의 진입 차수(indegree)를 계산한다.
- ② 진입 차수가 0인 정점들을 큐에 삽입한다.
- ③ 큐가 빌 때까지 다음을 반복:
 - a. 큐에서 정점을 꺼내 결과 리스트에 추가
 - b. 이 정점과 연결된 모든 간선 제거 → 연결된 정점들의 indegree를 1 감소
 - c. indegree가 0이 된 정점을 다시 큐에 삽입
- ④ 모든 정점을 방문한 후, 결과 리스트의 길이가 정점 수와 같다면 위상 정렬 성공 (그렇지 않다면 → 사이클 존재)

위상 정렬

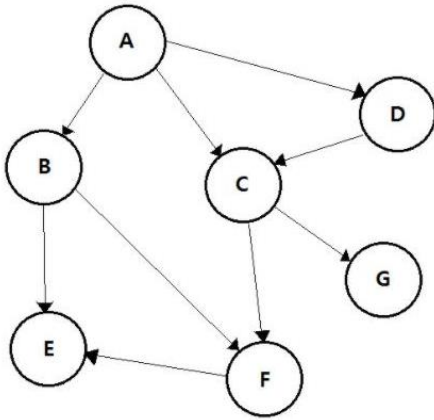
DFS를 이용한 위상 정렬

- 진출 차수를 활용한 정렬

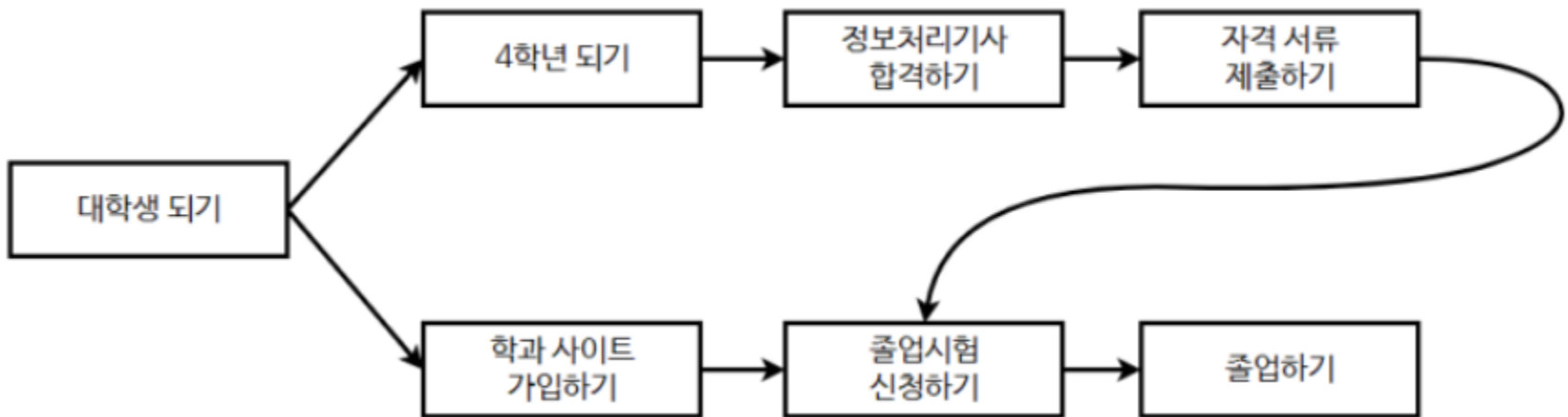
- ① visited 배열과 스택(stack 또는 리스트)을 준비
- ② 모든 정점에 대해:
 - 방문하지 않은 경우 DFS 수행
- ③ DFS에서는:
 - a. 현재 정점을 방문 표시
 - b. 인접한 정점들을 재귀적으로 DFS 수행
 - c. 인접한 정점 처리가 끝난 후, 현재 정점을 스택에 push
- ④ DFS가 끝난 후, 스택을 뒤집어 출력하면 위상 정렬 결과



위상 정렬



위상 정렬



진입 차수 알고리즘(배열)

```
1. Algorithm Compute_Indegree_Matrix(Matrix, Indegree)
2. Input: 인접 행렬 Matrix[n][n]
3. Output: 각 정점의 진입 차수 Indegree[]
4. for i <- 0 to n - 1 do {
5.     Indegree[i] <- 0
6. }
7. for j <- 0 to n - 1 do {
8.     for i <- 0 to n - 1 do {
9.         if (Matrix[i][j] = 1) {
10.            Indegree[j] <- Indegree[j] + 1
11.        }
12.    }
13. }
14. end
```

진입 차수 알고리즘(연결리스트)

1. Algorithm Compute_Indegree(List, Indegree)
2. Input: 그래프의 인접리스트 List
3. Output: 각 정점의 진입 차수 Indegree[]
4. for $i \leftarrow 0$ to $n - 1$ do {
5. Indegree[i] $\leftarrow 0$
6. }
7. for $u \leftarrow 0$ to $n - 1$ do {
8. $p \leftarrow \text{List}[u]$
9. while ($p \neq \text{Null}$) do {
10. Indegree[p.vertex] $\leftarrow \text{Indegree}[p.\text{vertex}] + 1$
11. $p \leftarrow p.\text{link}$
12. }
13. }
14. end

DFS 기반 위상 정렬 알고리즘

Algorithm DFS_Topological_Sort(Graph, n)

// 입력: 인접 리스트로 표현된 방향 그래프

Graph[0..n-1]

// 출력: 위상 정렬 결과를 stack 배열에 저장

1. for $i \leftarrow 0$ to $n - 1$ do

2. visited[i] $\leftarrow$ false

3. end for

4. top $\leftarrow -1$

5. procedure DFS(v)

6. visited[v] $\leftarrow$ true

7. p $\leftarrow$ Graph[v] 8. while p $\neq$ NULL do

9. u $\leftarrow$ p.vertex

10. if visited[u] = false then

11. DFS(u)

12. end if

13. p $\leftarrow$ p.link

14. end while

15. top $\leftarrow$ top + 1

16. stack[top] $\leftarrow$ v

17. end procedure

18. for $i \leftarrow 0$ to $n - 1$ do

19. if visited[i] = false then

20. DFS(i)

21. end if

22. end for

23. for $i \leftarrow$ top downto 0 do

24. print stack[i]

25. end for